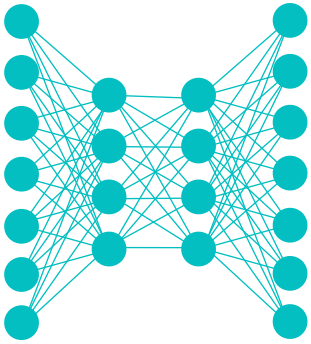
Lecture Notes for

Deep Learning

Encoder

Transformers



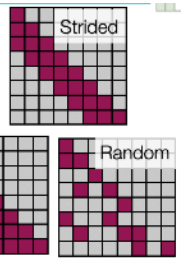
Logistics and Agenda

- Logistics
  - Grading
  - Paper Presentation Schedule Updated
- Agenda (probably two lectures)
  - Positional Encoding, RoPE
  - Student Paper Presentation
  - Common Transformers

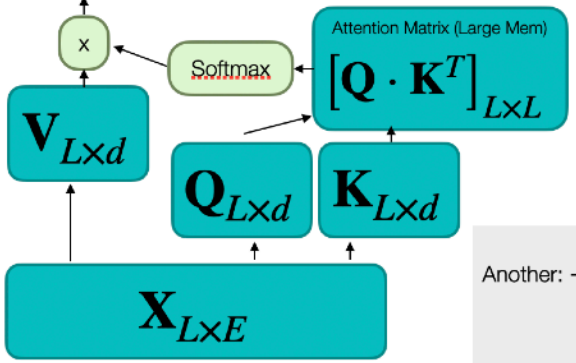
Last Time: Efficiency in Transformers

Naive Implementation:
- Computation: $O(L^2 \cdot d)$
- Memory: $O(L^2 + L \cdot d)$

One idea: limit non-zero values of $\mathbf{Q} \cdot \mathbf{K}^T$
Need to define sparsity before computation



$$\text{softmax}\left(\frac{\mathbf{Q} \cdot \mathbf{K}^T}{\sqrt{d}}\right) \cdot \mathbf{V}$$



Another idea: change softmax, to allow associative rule application

$$(\mathbf{Q} \cdot \mathbf{K}^T) \cdot \mathbf{V} = \mathbf{Q} \cdot (\mathbf{K}^T \cdot \mathbf{V})$$

$O(L^2 \cdot d)$ $\qquad$ $O(L \cdot d^2)$

Efficient Implementation:
- Computation: $O(L \cdot d^2)$
- Memory: $O(d^2 + L \cdot d)$

but we need a function that satisfies

$$f(\mathbf{Q} \cdot \mathbf{K}^T) = f(\mathbf{Q}) \cdot f(\mathbf{K}^T)$$

One function: softmax along rows and columns

$$\text{softmax}(\mathbf{Q}) \cdot (\text{softmax}(\mathbf{K})^T \cdot \mathbf{V})$$

Katharopoulos et al., **Trans are RNNs**, ICLR 2021

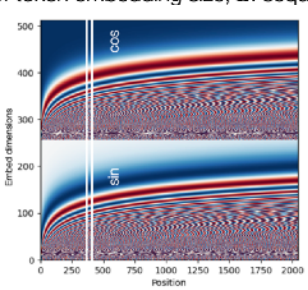
Another: $\frac{\mathbf{Q}}{\|\mathbf{Q}\|} \cdot \left(\frac{\mathbf{K}}{\|\mathbf{K}\|}^T \cdot \mathbf{V}\right) \rightarrow \frac{\mathbf{Q} \cdot \mathbf{K}^T}{\|\mathbf{Q}\| \|\mathbf{K}\|} \cdot \mathbf{V}$

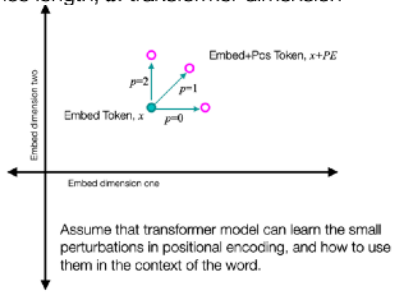
same as cosine similarity

Mongaras, Dohm, and Larson, **Cottention**, CC 2025

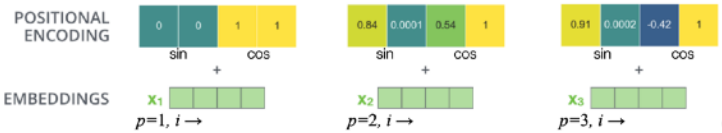
$\boldsymbol{E}$: token embedding size, $\boldsymbol{L}$: sequence length, $\boldsymbol{d}$: transformer dimension

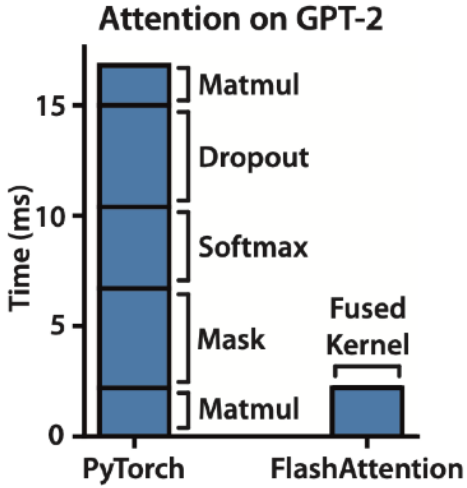
44

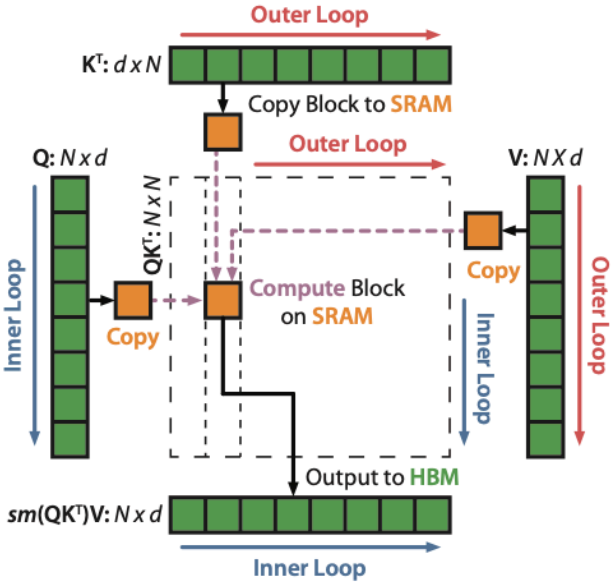




Assume that transformer model can learn the small perturbations in positional encoding, and how to use them in the context of the word.







FlashAttention

RoPE

SAY VIBE CODING ONE MORE TIME
What can I help you ship?
What do you want to build?
What can I help you ship?
Idea to app in seconds.
What can I help you creat
What do you want to build today?
Hey! What you want to Build?
What do you want to build?
Mobile apps in minutes.
What do you want to build today?
Create it for free
Where ideas begin

Relative Position

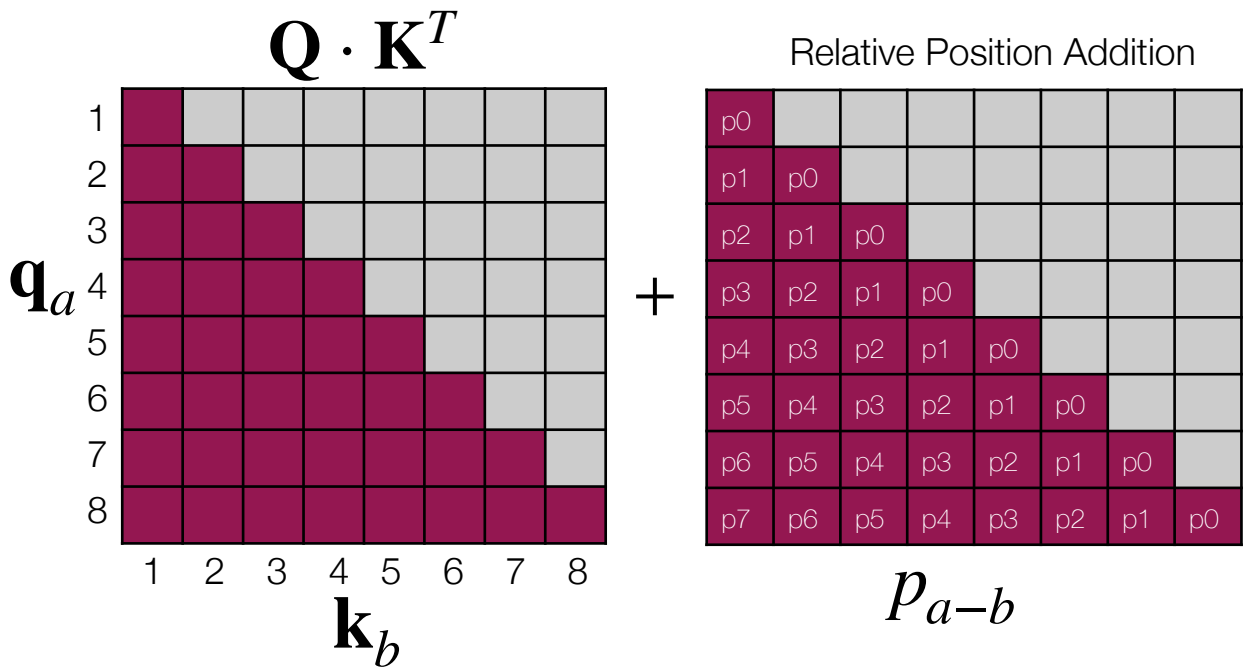
The old house sat silently at the end of the street, its windows like empty eyes watching the neighborhood. Sarah hesitantly pushed the rusty gate open, the screech of metal echoing in the quiet afternoon. She had promised her friends she would get the old ball back, but now she wasn't so sure.

Absolute Position Encoding: The word "house" occurs at position three.
(+) Easy to encode
(+) Not too memory intensive (just adds to word embedding)
(—) Forces transformer to learn word relationships from raw position.

Relative Position Encoding: The word "house" occurs directly after "old" and two positions before "silent."
(+) Seems like this is more actionable and usable in language processing.
(—) How on earth can we do this?
(—) How on earth can we do this, ***efficiently***?

Relative Positional Encoding

- Don't encode raw position, encode relative position between $\mathbf{q}_a$ and $\mathbf{k}_b$ at position a and b
- practically, attention → softmax($\mathbf{Q} \cdot \mathbf{K}^T + p_{a-b}$)



- (+) Nicely structured relative position information
- (-) Lots more memory
- (-) Couples position and attention (harder to distribute ops)

How might we still encode relative position, without all the overhead?

Smart relative position encoding

- *Ideally*, for sequence $\left[\mathbf{x}_1, \mathbf{x}_2, \ldots \mathbf{x}_a, \ldots \mathbf{x}_b, \ldots \mathbf{x}_L\right]$, relative position attention (positions a and b) is given by:

$$f(\mathbf{x}_a, \mathbf{x}_b, a-b) = \left(\underbrace{\mathbf{W}^{(q)} \cdot \mathbf{x}_a}_{\mathbf{q}_a}\right)^T R_{a-b} \left(\underbrace{\mathbf{W}^{(k)} \cdot \mathbf{x}_b}_{\mathbf{k}_b}\right)$$

$$= \mathbf{q}_a^T \cdot R_{a-b} \cdot \mathbf{k}_b$$

$$= \mathbf{q}_a^T \cdot R_a R_b \cdot \mathbf{k}_b$$

When Multiplied, still encodes relative pos

$$= (\mathbf{q}_a^T \cdot R_a)(R_b \cdot \mathbf{k}_b) = \hat{\mathbf{q}}_a^T \cdot \hat{\mathbf{k}}_b$$

Still about as Efficient as Attention

Precompute

$$(\mathbf{q}_a \cdot R_a) \rightarrow \hat{\mathbf{q}}_a$$

$$(R_b \cdot \mathbf{k}_b) \rightarrow \hat{\mathbf{k}}_b$$

$$\text{softmax}(\hat{\mathbf{q}}_a^T \cdot \hat{\mathbf{k}}_b)$$

$$\text{softmax}\left(f(\mathbf{x}_a, \mathbf{x}_b, a-b)\right)$$

relative position attention

- what if $\boxed{R_{a-b}}$ can be decoupled into $\boxed{R_a R_b}$ for efficiency?

$$R_a = e^{j \cdot \theta a} \qquad R_b = e^{-j \cdot \theta b} \qquad R_a R_b = e^{j \cdot \theta (a-b)} = R_{a-b}$$

one solution: rotations in the complex plain, eigenfunctions

Smart relative position encoding

a, b = position
R = 2D matrix

- but practically $R_a R_b = e^{j \cdot \theta(a-b)} = R_{a-b}$ requires complex valued arithmetic (not ideal for efficiency). So we can get the same benefit via:

$$f(\mathbf{x}_a, \mathbf{x}_b, a-b) = \text{Re}[\mathbf{q}_a^T \cdot R_a R_b \cdot \mathbf{k}_b] = \mathbf{q}_a^T \cdot \text{Re}[R_a R_b] \cdot \mathbf{k}_b$$

- which in the 2 element case reduces to the rotation matrix:

$$R_a \cdot \mathbf{q}_a = \begin{bmatrix} \cos(a \cdot \theta) & -\sin(a \cdot \theta) \\ \sin(a \cdot \theta) & \cos(a \cdot \theta) \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} \qquad R_b \cdot \mathbf{k}_b = \begin{bmatrix} \cos(b \cdot \theta) & -\sin(b \cdot \theta) \\ \sin(b \cdot \theta) & \cos(b \cdot \theta) \end{bmatrix} \begin{bmatrix} k_1 \\ k_2 \end{bmatrix}$$

Rotate $\boldsymbol{q}$ by $a\,\theta$, precomputed — Rotate $\boldsymbol{k}$ by $b\,\theta$, precomputed

- and we effectively get relative position encoding for $(\mathbf{q}_a^T \cdot R_a)(R_b \cdot \mathbf{k}_b)$, but with decoupled operations!!!
- but, expanding beyond 2D vectors starts to make the operation too computational… so let's only do operation in pairs along $\mathbf{q}$ and along $\mathbf{k}$ separately (lots of 2D rotations)

Rotary Position Encoding, RoPE

a, b = position
R = 2D matrix

- Now we can finally understand RoPE:

$$R_a = \begin{bmatrix} \cos(a\cdot\theta_1) & -\sin(a\cdot\theta_1) & 0 & 0 & \dots & 0 & 0 \\ \sin(a\cdot\theta_1) & \cos(a\cdot\theta_1) & 0 & 0 & \dots & 0 & 0 \\ \vdots & \vdots & \cos(a\cdot\theta_2) & -\sin(a\cdot\theta_2) & \dots & \vdots & \vdots \\ \vdots & \vdots & \sin(a\cdot\theta_2) & \cos(a\cdot\theta_2) & \dots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 0 & 0 & \cos(a\cdot\theta_{D/2}) & -\sin(a\cdot\theta_{D/2}) \\ 0 & 0 & \dots & 0 & 0 & \sin(a\cdot\theta_{D/2}) & \cos(a\cdot\theta_{D/2}) \end{bmatrix}$$

Rotate by $a \cdot \theta_1$ *(large rotate)*

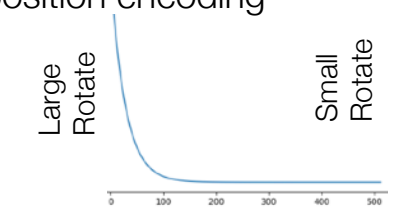
Rotate by $a \cdot \theta_2$

Rotate by $a \cdot \theta_{D/2}$ *(small rotate)*

In general, produces **better results** (mostly) while being **not too computational** (a "pretty good" relative encoding)

Lots of pairwise rotations, each preserving the property of relative position encoding

$\theta_i = 10000^{-2(i-1)/D}$ defines range of rotations, D is dims of q,k



High frequency, sensitive to position

Speed: Fast to implement with two point wise vector multiplies and addition (low overhead, still parallel)

$$R_a \cdot \mathbf{q}_a = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ \vdots \\ q_{D-1} \\ q_D \end{bmatrix} \cdot \begin{bmatrix} \cos(a\cdot\theta_1) \\ \cos(a\cdot\theta_1) \\ \cos(a\cdot\theta_2) \\ \cos(a\cdot\theta_2) \\ \vdots \\ \cos(a\cdot\theta_{D/2}) \\ \cos(a\cdot\theta_{D/2}) \end{bmatrix} + \begin{bmatrix} -q_2 \\ q_1 \\ -q_4 \\ q_3 \\ \vdots \\ -q_D \\ q_{D-1} \end{bmatrix} \cdot \begin{bmatrix} \sin(a\cdot\theta_1) \\ \sin(a\cdot\theta_1) \\ \sin(a\cdot\theta_2) \\ \sin(a\cdot\theta_2) \\ \vdots \\ \sin(a\cdot\theta_{D/2}) \\ \sin(a\cdot\theta_{D/2}) \end{bmatrix}$$

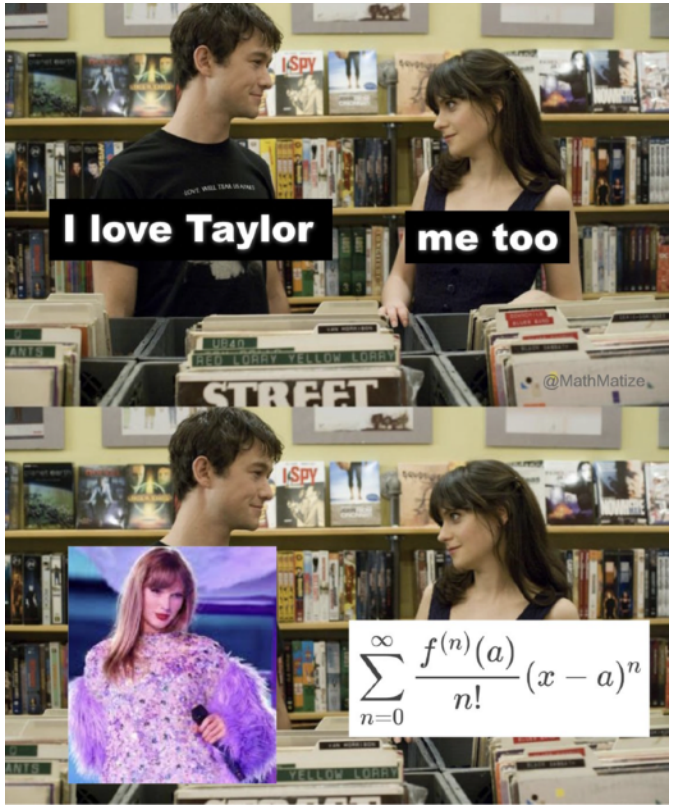
can pre-compute for each position a

Transformer learns to encode positionally sensitive aspects in high frequency indices…

Low frequency, less sensitive to position

Su et al., RoFormer, ACM Neurocomputing (2023) https://arxiv.org/pdf/2104.09864

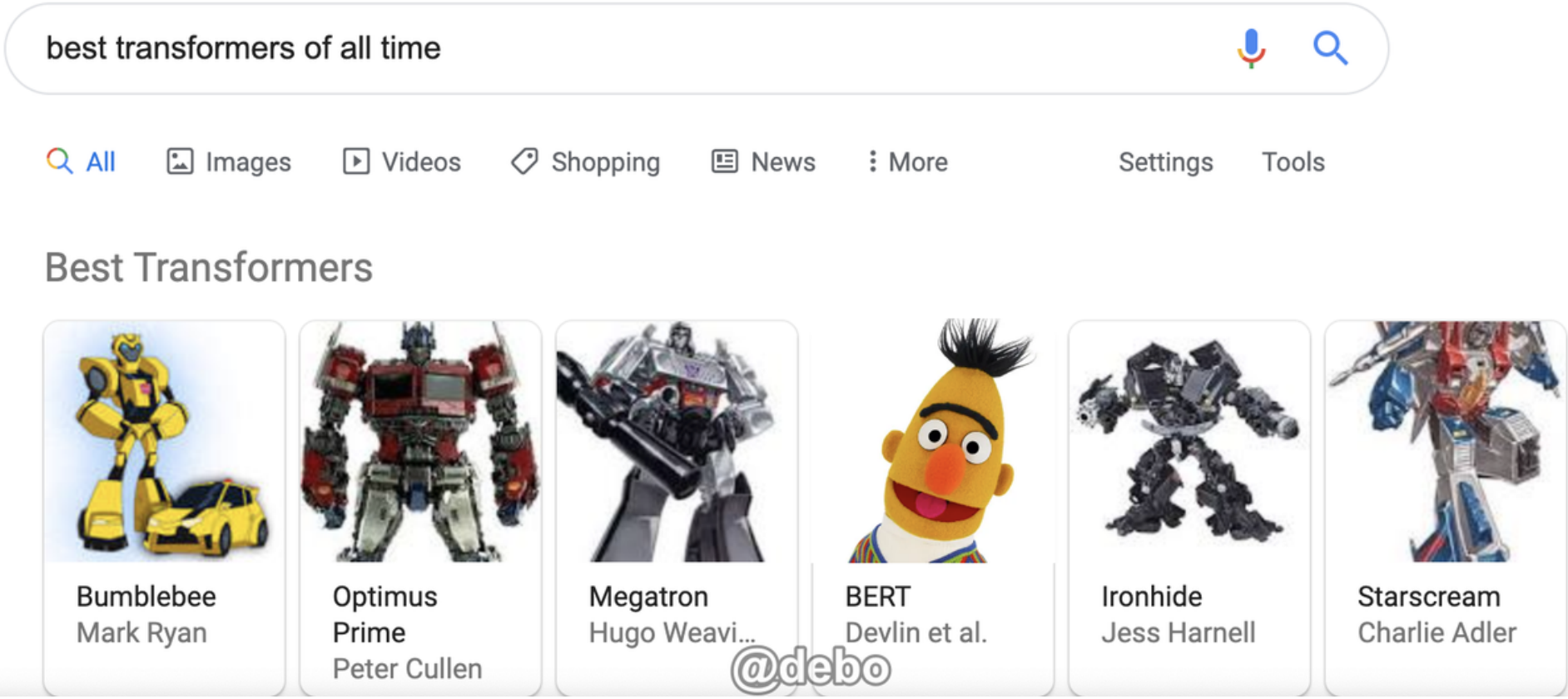
Paper Presentation



$$
\begin{aligned}
O_t &= G_t \sum_{s=1}^{t} e^{Q_t \cdot K_s^T} V_s \\
&= G_t \sum_{s=1}^{t} \sum_{n=0}^{\infty} \frac{1}{n!} (Q_t \cdot K_s^T)^n V_s \\
&= G_t \sum_{s=1}^{t} \sum_{n=0}^{\infty} \frac{1}{n!} \sum_{i=1}^{d^n} \left(Q_t^{\otimes n}\right)_i \left((K_s^{\otimes n})^T\right)_i V_s \\
&= G_t \sum_{n=0}^{\infty} \frac{1}{n!} \sum_{i=1}^{d^n} \sum_{s=1}^{t} \left(Q_t^{\otimes n}\right)_i \left((K_s^{\otimes n})^T\right)_i V_s \\
&= G_t \sum_{n=0}^{\infty} \frac{1}{n!} \sum_{i=1}^{d^n} \left(Q_t^{\otimes n}\right)_i \sum_{s=1}^{t} \left((K_s^{\otimes n})^T\right)_i V_s \\
&= G_t \sum_{n=0}^{\infty} \frac{1}{n!} \left(Q_t^{\otimes n}\right) \sum_{s=1}^{t} \left((K_s^{\otimes n})^T\right) \cdot V_s \\
&= G_t \sum_{n=0}^{\infty} \frac{1}{n!} (Q_t^{\otimes n}) H_t^n, \qquad H_t^n = \sum_{s=1}^{t} ((K_s^{\otimes n})^T) \cdot V_s
\end{aligned}
$$

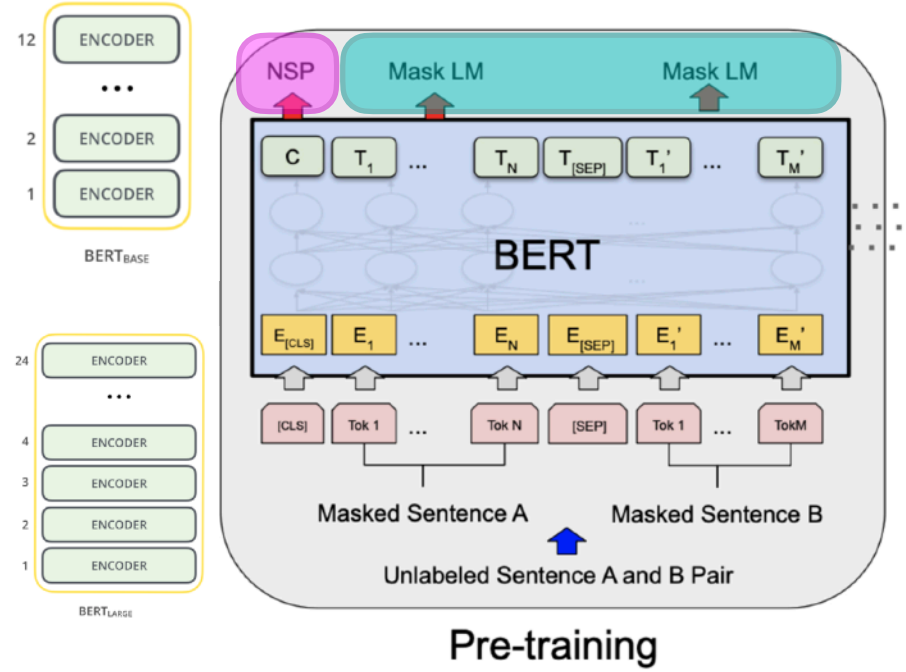
Taylor Swift: Fast Inference Softmax through Recurrent Taylor Expansion

Encoder Transformers



Bidirectional Encoder Representation

- Google, 2018. Vocab: 30k words
- Bidirectional (non-causal attention)
- $BERT_{Base}$
  - 12 encoder layers, 12 heads/layer
  - 110M parameters
- $BERT_{Large}$
  - 24 encoder layers, 16 heads/layer
  - 340M parameters



Masked Language Modeling (Mask LM)

"I am **[MASK1]** in CS5/7-321 at SMU. This class is **[MASK2]**"

MASK1: **"enrolled"** MASK2: **"great"**

Next Sentence Prediction (NSP)

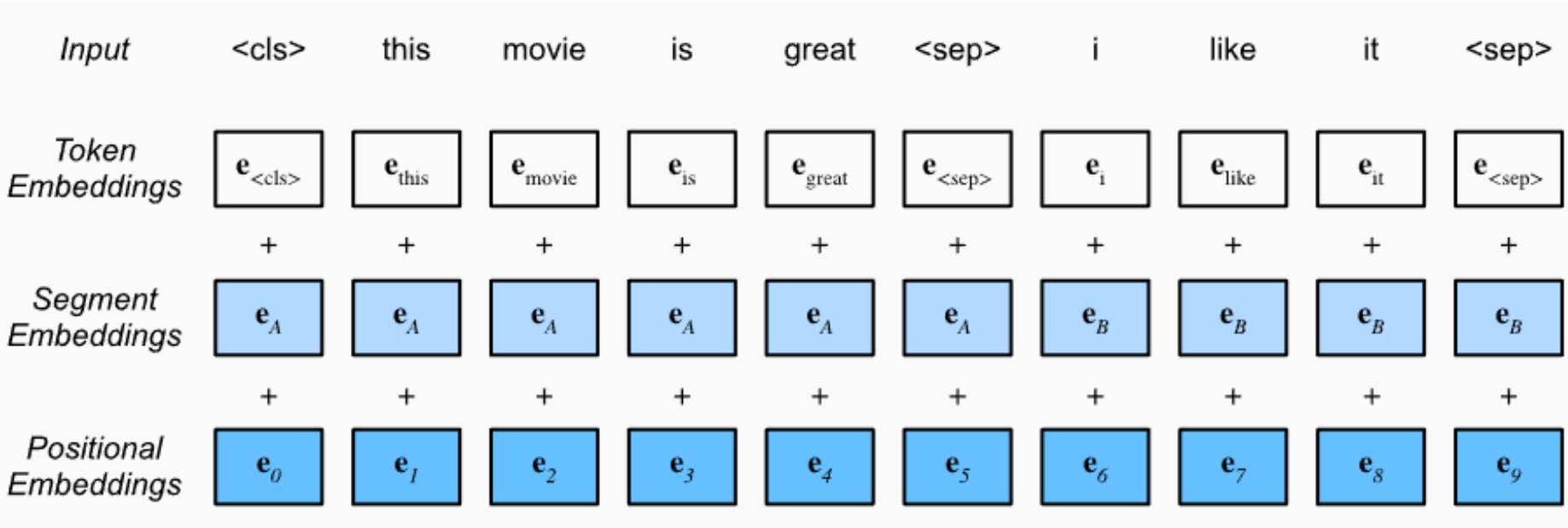
"[CLS] Dr. Larson is a professor [SEP] his class examples are great" → **Label "IsNext"**

"[CLS] Dr. Larson is a professor [SEP] do you like bread" → **Label "NotNext"**

Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". arXiv:1810.04805v2

BERT Embeddings

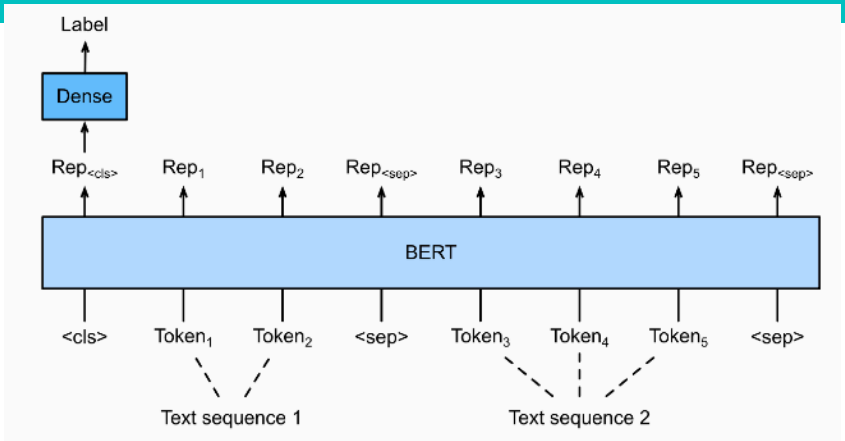
- Three embeddings are used:
  - Token: 30,522 → 768
  - Position (trained): 512 → 768
  - Segment (useless): 2 → 768

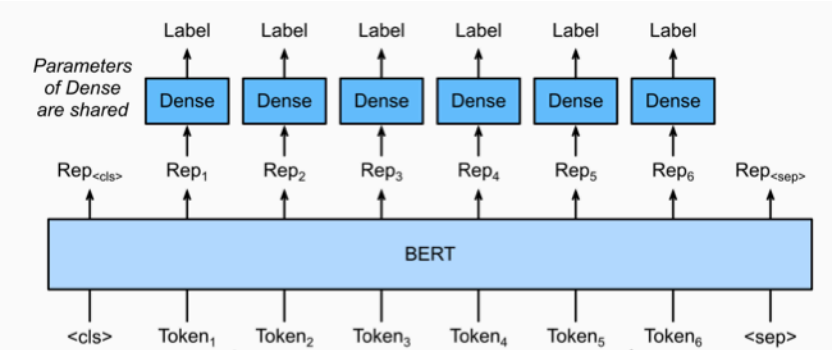


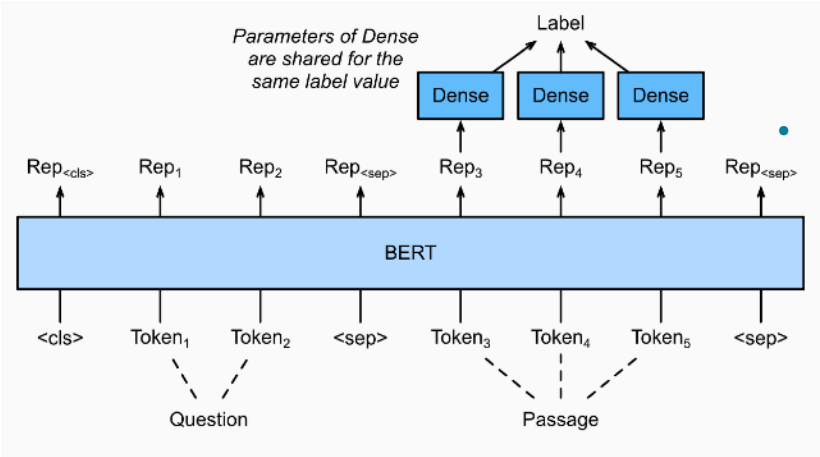
Fine Tuning BERT

- Sentence predict: like Text Similarity
  - Make use of NSP
  - Two sentences, do they belong?

- Part of speech tagging
  - Make use of Masked LM
  - Shared dense layer for each Rep







Question Answering (Stanford QA Dataset, SQuaD)

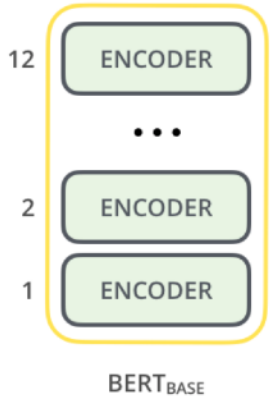
- Make use of Masked LM
- Highlight passage text that answers given question

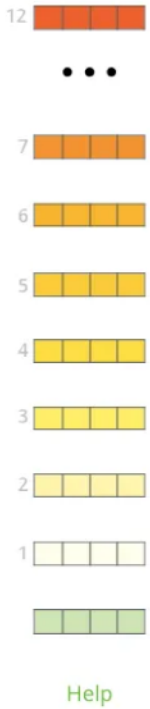
Q: Who currently teaches machine learning at SMU?
P: "Machine learning was first offered at SMU in the 1990's. **Dr. Larson** has been teaching the course since 2014 and has changed it into a neural networks course, despite its origins. "

https://classic.d2l.ai/chapter_natural-language-processing-applications/finetuning-bert.html

Fine Tuning BERT

- Could we use more than just the final output layers?

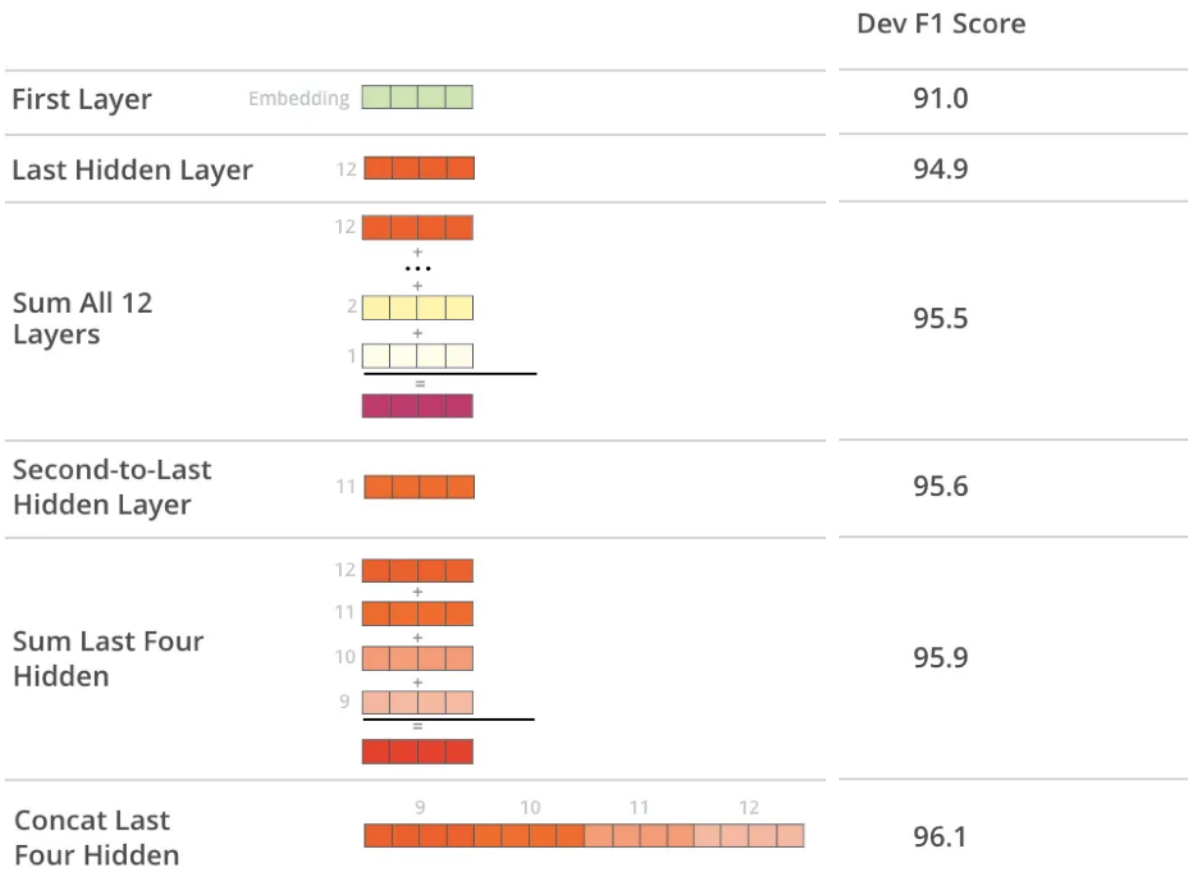




Each word has 12 layer outputs in $BERT_{Base}$

In what context is the word "help" used?

Noun, verb, adverb, etc.



https://jalammar.github.io/illustrated-bert/

MELVA Results (my lab)

- Measuring English Language Vocabulary Acquisition
- Or results from my lab:
  - Using science terms in sentence?
  - Collect/transcribe responses
- Collected about 6000 sentences

- Transfer learn based upon LM output
  - Without transformer encoder: ~75%
  - With transformer encoder: ~84%

L@S '23, July 20–22, 2023, Copenhagen, Denmark

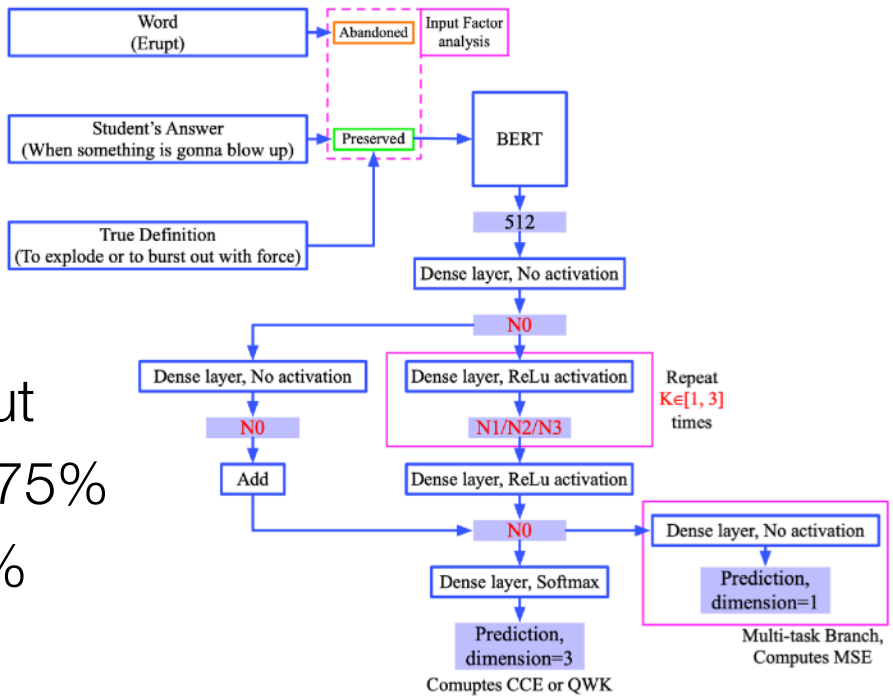


Figure 1: Example of the end-to-end pipeline of the network. Variables marked in red are found through hyperparameter search.

Zhongdi Wu, Larson, E., Makoto Sano, Doris Baker, Akihito Kamata, & Nathan Gage (2023) Towards Scalable Vocabulary Acquisition Assessment with BERT. Learning at Scale, 5. 10.1145/3573051.3596170